```
/**
 * ============================================================
 * Uluka Ultra © — Master Licence & Intelligence Server
 * Version: 5.1 — Claude Edition
 * ============================================================
 * CHANGE IN v5.1 vs v5.0:
 * - doGet() fixed: renamed 'type' → 'reqType' (was causing
 *   "cannot access before initialisation" error)
 * - Dashboard routing added: ?type=dashboard serves HTML
 * - doPost() same fix applied for safety
 * - All other functions IDENTICAL to v5.0
 * ============================================================
 */

// ============================================================
// 1. CONFIGURATION
// ============================================================

const props           = PropertiesService.getScriptProperties();
const MASTER_SHEET_ID = SpreadsheetApp.getActiveSpreadsheet().getId();
const SHEET_NAME      = "Key_Database";

const TELEGRAM_BOT_TOKEN  = props.getProperty('TG_TOKEN');
const ADMIN_CHAT_ID       = props.getProperty('ADMIN_ID');
const PREMIUM_GROUP_ID    = props.getProperty('PREMIUM_GROUP_ID');
const FREE_GROUP_ID       = props.getProperty('FREE_GROUP_ID');
const MAKE_WEBHOOK_URL    = props.getProperty('MAKE_URL');
const DISCLAIMER          = props.getProperty('DISCLAIMER') ||
                            "⚠️ Performance demonstration only. NOT financial
advice.";

const COLUMNS = {
  CLIENT_NAME:      1,
  LICENCE_KEY:      2,
  HWID:             3,
  ACCOUNT_ID:       4,
  CREATION_DATE:    5,
  SUBSCRIPTION:     6,
```

```
  DURATION:         7,
  EXPIRES_ON:       8,
  STATUS:           9,
  EQUITY_CAP:      10,
  LAST_SYNC:       11,
  BLACKLISTED:     12,
  ACTIVATIONS:     13,
  INSTANCE_ID:     14,
  PERSONAL_CHAT_ID:15
};

// ============================================================
// 2. UI & AUTO-SETUP
// ============================================================

function onOpen() {
  SpreadsheetApp.getUi().createMenu('🦉 Uluka Admin')
    .addItem('🦉 Generate New Key',   'showKeyGeneratorPrompt')
    .addItem('📡 Send Test Alert',     'sendTestAlert')
    .addItem('🔓 Clear HWID Binding',  'clearHwidData')
    .addItem('⚙️ Setup All Triggers',  'setupAllSystemTriggers')
    .addItem('🔍 System Health Check', 'hourlyMaintenance')
    .addItem('📊 Send EOD Report Now', 'sendDailyEODReport')
    .addToUi();
}

function setupAllSystemTriggers() {
  ScriptApp.getProjectTriggers().forEach(t => ScriptApp.deleteTrigger(t));

  ScriptApp.newTrigger("sendDailyEODReport")
    .timeBased().everyDays(1).atHour(23).nearMinute(55).create();
  ScriptApp.newTrigger("autoCheckExpiry")
    .timeBased().everyDays(1).atHour(1).create();
  ScriptApp.newTrigger("hourlyMaintenance")
    .timeBased().everyHours(1).create();
  ScriptApp.newTrigger("runFridayPayeArchive")

.timeBased().onWeekDay(ScriptApp.WeekDay.FRIDAY).atHour(23).nearMinute(30).create();
```

```
  SpreadsheetApp.getUi().alert(
    "✅ All Triggers Set!\n\n" +
    "• EOD Report: Daily 23:55\n" +
    "• Expiry Check: Daily 01:00\n" +
    "• Maintenance: Every Hour\n" +
    "• PAYE Archive: Friday 23:30"
  );
}

function hourlyMaintenance() {
  autoCheckExpiry();
  const ss = SpreadsheetApp.getActiveSpreadsheet();
  const logSheet = ss.getSheetByName("RateLimit");
  if (logSheet && logSheet.getLastRow() > 500) {
    const rowsToDelete = logSheet.getLastRow() - 100;
    logSheet.deleteRows(2, rowsToDelete);
    sendAdminAlert("🧹 <b>Maintenance:</b> Cleared " + rowsToDelete + " old log
entries.");
  }
}

// ==============================================================
// 3. WEB COMMUNICATION — doGet + doPost
// ==============================================================

function doGet(e) {
  try {
    const params  = e.parameter || {};
    const reqType = (params.type || "").toLowerCase();  // ← FIXED: was 'type', now
'reqType'
    const accId   = params.account || params.account_id || "";

    // ── Dashboard HTML ──
    if (reqType === "dashboard") {
  const tpl = HtmlService.createTemplateFromFile('dashboard');
  tpl.gasUrl = ScriptApp.getService().getUrl();   // ← injects real exec URL
  return tpl.evaluate()
```

```
    .setTitle('🦉 Uluka Ultra — Live Dashboard')
    .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}

    // ── Dashboard Live Data API ──
    if (reqType === "dashboard_data") {
      return getDashboardDataForClient();
    }

    // ── Cloud config ──
    if (reqType === "get_config") return getCloudConfig();

    // ── Licence validation ──
    if (reqType === "validate" || reqType === "auth") return
validateLicence(params);

    // ── Kill switch check for other requests ──
    if (accId) {
      const status = checkKillSwitch(accId);
      if (status !== "allowed")
        return ContentService.createTextOutput(status.toUpperCase());
    }

    return ContentService.createTextOutput("ULUKA_SERVER_OK");

  } catch (err) {
    console.error("doGet error: " + err.message);
    return ContentService.createTextOutput("SERVER_ERROR: " + err.message);
  }
}

function doPost(e) {
  try {
    const data    = JSON.parse(e.postData.contents);
    const acc     = (data.account || data.account_id || "unknown").toString();
    const reqType = (data.type || data.msg_type || "").toString(); // ← FIXED: was
'type'
```

```
  logRateLimit("POST", acc);
  processBillingSync(data);

  if (reqType === "TRADE_SIGNAL" || reqType === "🦉 Uluka Live Hoot") {
    sendLiveHootToGroups(data);
    return ContentService.createTextOutput("HOOT_SENT");
  }

  if (reqType === "TRADE_LOG") {
    logTradeToSheet(data);
    return ContentService.createTextOutput("LOG_OK");
  }

  if (reqType === "ActivationAlert") {
    sendAdminAlert("🆕 <b>EA ACTIVATION</b>\n" + data.text);
    sendGroupActivationHoot(data);
    return ContentService.createTextOutput("ACTIVATION_OK");
  }

  if (reqType === "GuardianAlert") {
    handleGuardianAlert(data);
    return ContentService.createTextOutput("GUARDIAN_OK");
  }

  if (reqType === "ClientEOD") {
    sendClientEODReport(data);
    return ContentService.createTextOutput("EOD_OK");
  }

  if (reqType === "TradeCard")
    return ContentService.createTextOutput("CARD_OK");

  return ContentService.createTextOutput("SUCCESS");

} catch (err) {
  console.error("doPost error: " + err.message);
  return ContentService.createTextOutput("ERROR: " + err.message);
}
```

```
}

// ============================================================
// 4. CLOUD CONFIG
// ============================================================

function getCloudConfig() {
  const ss       = SpreadsheetApp.getActiveSpreadsheet();
  const cfgSheet = ss.getSheetByName("Uluka_Cloud");

  if (!cfgSheet) {
    return ContentService
      .createTextOutput(JSON.stringify({
        "multiplier":     "1.0",
        "min_confidence": "65",
        "news_filter":    "ON",
        "kill_switch":    "OFF"
      }))
      .setMimeType(ContentService.MimeType.JSON);
  }

  const row    = cfgSheet.getRange("A2:D2").getValues()[0];
  const config = {
    "multiplier":     (row[0] || "1.0").toString().trim(),
    "min_confidence": (row[1] || "65").toString().trim(),
    "news_filter":    (row[2] || "ON").toString().trim().toUpperCase(),
    "kill_switch":    (row[3] || "OFF").toString().trim().toUpperCase()
  };

  return ContentService
    .createTextOutput(JSON.stringify(config))
    .setMimeType(ContentService.MimeType.JSON);
}

// ============================================================
// 5. LICENCE VALIDATION
// ============================================================
```

```
function validateLicence(p) {
  try {
    const ss    = SpreadsheetApp.getActiveSpreadsheet();
    const sheet = ss.getSheetByName(SHEET_NAME);
    const data  = sheet.getDataRange().getValues();
    const keyIn = (p.key     || "").trim();
    const accIn = (p.account || "").trim();
    const broker= (p.broker  || "").trim();

    for (let i = 1; i < data.length; i++) {
      const sheetKey = String(data[i][COLUMNS.LICENCE_KEY - 1]).trim();
      if (sheetKey !== keyIn) continue;

      const status = String(data[i][COLUMNS.STATUS - 1]).toUpperCase();
      if (status !== "ACTIVE")
        return ContentService.createTextOutput("BLACKLISTED");

      if (accIn) {
        const killStatus = checkKillSwitch(accIn);
        if (killStatus !== "allowed")
          return ContentService.createTextOutput(killStatus.toUpperCase());
      }

      const boundAcc = String(data[i][COLUMNS.ACCOUNT_ID - 1]).trim();
      if (boundAcc !== "" && accIn !== "" && boundAcc !== accIn) {
        sendAdminAlert(
          "⚠️ <b>ACCOUNT MISMATCH</b>\n" +
          "Key: <code>" + sheetKey + "</code>\n" +
          "Bound: <code>" + boundAcc + "</code>\n" +
          "Incoming: <code>" + accIn + "</code>"
        );
        return ContentService.createTextOutput("ACCOUNT_MISMATCH");
      }

      if (accIn && p.balance) processBillingSync(p);

      // —— Expiry Parse (v5.3 fix: guard against empty/invalid cell) ——
      const rawExpiryVal = data[i][COLUMNS.EXPIRES_ON - 1];
```

```
      let expiryDate;

      if (rawExpiryVal instanceof Date && !isNaN(rawExpiryVal.getTime())) {
        // Cell is already a Sheets Date object — use directly
        expiryDate = new Date(rawExpiryVal);
      } else {
        const rawExpiry = (rawExpiryVal || "").toString().trim();
        if (!rawExpiry) {
          // Empty cell — no expiry set yet, treat as not expired (1 year default)
          expiryDate = new Date();
          expiryDate.setFullYear(expiryDate.getFullYear() + 1);
        } else {
          const ep = rawExpiry.split("-");
          if (ep.length === 3 && ep[0].length === 2) {
            // DD-MM-YYYY format
            expiryDate = new Date(ep[2] + "/" + ep[1] + "/" + ep[0]);
          } else {
            expiryDate = new Date(rawExpiry);
          }
        }
      }

      // Final guard: if still invalid, fallback to 1 year from now
      if (isNaN(expiryDate.getTime())) {
        console.warn("Invalid expiry for key: " + sheetKey + " raw: " +
String(rawExpiryVal));
        expiryDate = new Date();
        expiryDate.setFullYear(expiryDate.getFullYear() + 1);
      }

      expiryDate.setHours(23, 59, 59, 999);
      const formattedDate = Utilities.formatDate(expiryDate, "GMT", "dd-MM-yyyy");

      const bSheet = ss.getSheetByName("Billing");
      let actualDeposit = p.balance || "0.00";
      if (bSheet && accIn) {
        const bData = bSheet.getDataRange().getValues();
        for (let j = 1; j < bData.length; j++) {
```

```
          if (String(bData[j][1]) === accIn) {
            actualDeposit = bData[j][7] || actualDeposit;
            break;
          }
        }
      }

      let acts = parseInt(data[i][COLUMNS.ACTIVATIONS - 1]) || 0;

      if (p.instance && p.instance !== data[i][COLUMNS.INSTANCE_ID - 1]) {
        acts++;
        sheet.getRange(i + 1, COLUMNS.ACTIVATIONS).setValue(acts);
        sheet.getRange(i + 1, COLUMNS.INSTANCE_ID).setValue(p.instance);
        if (p.hwid)             sheet.getRange(i + 1,
COLUMNS.HWID).setValue(p.hwid);
        if (accIn)              sheet.getRange(i + 1,
COLUMNS.ACCOUNT_ID).setValue(accIn);
        if (p.personal_chat_id) sheet.getRange(i + 1,
COLUMNS.PERSONAL_CHAT_ID).setValue(p.personal_chat_id);

        const clientName = data[i][0] || "Client";
        const equityCap  = data[i][COLUMNS.EQUITY_CAP - 1];

        sendAdminAlert(
          `🆕 <b>NEW ACTIVATION [${acts}/30]</b>\n` +
          `👤 <b>Client:</b> ${clientName}\n` +
          `🔑 <b>Key:</b> <code>${sheetKey}</code>\n` +
          `🆔 <b>Account:</b> <code>${accIn || "N/A"}</code>\n` +
          `🏦 <b>Broker:</b> ${broker || "Unknown"}\n` +
          `💰 <b>Deposit:</b> $${actualDeposit}\n` +
          `🛑 <b>Tier Cap:</b> $${equityCap}\n` +
          `📅 <b>Expires:</b> ${formattedDate}\n` +
          `⏱️ <b>Time:</b> ${Utilities.formatDate(new Date(), "GMT+5:30",
"dd-MMM-yyyy HH:mm")}`
        );

        const clientPersonalId = data[i][COLUMNS.PERSONAL_CHAT_ID - 1] ||
p.personal_chat_id || "";
```

```
        if (clientPersonalId) {
          sendToChat(clientPersonalId,
            `🦉 <b>ULUKA ULTRA — ACTIVATED!</b>\n\n` +
            `Welcome, <b>${clientName}</b>!\n\n` +
            `✅ Your EA is now live and trading.\n` +
            `📅 Licence expires: <b>${formattedDate}</b>\n` +
            `🏆 You will receive your daily EOD reports here.\n\n` +
            `<i>${DISCLAIMER}</i>`
          );
        }
      }

      if (acts > 30)
        return ContentService.createTextOutput("LIMIT_REACHED");

      sheet.getRange(i + 1, COLUMNS.LAST_SYNC).setValue(new Date());

      return ContentService.createTextOutput(
        "AUTHORIZED|" + formattedDate + "|" + (data[i][0] || "Client")
      );
    }

    return ContentService.createTextOutput("NOT_FOUND");

  } catch (err) {
    console.error("validateLicence error: " + err.message);
    return ContentService.createTextOutput("SERVER_ERROR: " + err.message);
  }
}

// ============================================================
// 6. KILL SWITCH (Monday block removed)
// ============================================================

function checkKillSwitch(accountId) {
  try {
    const bSheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("Billing");
    if (!bSheet) return "allowed";
```

```
    const data = bSheet.getDataRange().getValues();

    for (let i = 1; i < data.length; i++) {
      if (String(data[i][1]).trim() !== String(accountId).trim()) continue;

      const status = (data[i][6] || "").toString().toLowerCase();
      const share  = parseFloat(data[i][5]) || 0;
      const limit  = parseFloat(data[i][9]) || 50.0;

      if (status.includes("blocked")) return "blocked";
      if (share >= limit)             return "limit_block";

      return "allowed";
    }

    return "allowed";

  } catch (err) {
    console.error("checkKillSwitch error: " + err.message);
    return "allowed";
  }
}

// ==============================================================
// 7. BILLING SYNC
// ==============================================================

function processBillingSync(p) {
  try {
    const ss = SpreadsheetApp.getActiveSpreadsheet();
    let bSheet = ss.getSheetByName("Billing");

    if (!bSheet) {
      bSheet = ss.insertSheet("Billing");
      bSheet.appendRow([
        "Client","Account ID","Start Balance","Current Balance",
        "Net Profit","Payee 25%","Status","Initial Equity",
```

```
      "Drawdown %","Payee Limit","Last Sync","Broker"
    ]);
  }

  const accId      = String(p.account || p.account_id || "").trim();
  const currentBal = parseFloat(p.balance || 0);
  const currentEq  = parseFloat(p.equity  || currentBal);
  const brokerName = p.broker  || "";
  const clientName = p.client  || "";

  if (!accId) return;

  const bData = bSheet.getDataRange().getValues();

  for (let i = 1; i < bData.length; i++) {
    if (String(bData[i][1]).trim() !== accId) continue;

    bSheet.getRange(i+1, 4).setValue(currentBal);
    bSheet.getRange(i+1, 5).setFormula(`=D${i+1}-C${i+1}`);
    bSheet.getRange(i+1, 6).setFormula(`=IF(E${i+1}>0,E${i+1}*0.25,0)`);

    const initial = parseFloat(bData[i][7]) || currentBal;
    const dd      = initial > 0 ? ((initial - currentEq) / initial * 100) : 0;
    bSheet.getRange(i+1, 9).setValue(dd.toFixed(2) + "%");
    bSheet.getRange(i+1, 11).setValue(new Date());
    if (brokerName) bSheet.getRange(i+1, 12).setValue(brokerName);
    return;
  }

  bSheet.appendRow([
    clientName || "New Client", accId, currentBal, currentBal,
    0, 0, "🟢 Active", currentBal, "0.00%", 50, new Date(), brokerName
  ]);

 } catch (err) {
  console.error("processBillingSync error: " + err.message);
 }
}
```

```
// ============================================================
// 8. LIVE HOOT TO GROUPS
// ============================================================

function sendLiveHootToGroups(data) {
  const time   = data.time   || Utilities.formatDate(new Date(), "GMT+2",
"HH:mm:ss");
  const action = (data.action || "BUY").toUpperCase();
  const icon   = action === "BUY" ? "🟢" : "🔴";
  const strat  = data.strategy || "Uluka Vision";
  const sess   = data.session  || "";
  const conf   = data.confidence || data.conf || "";
  const health = data.health !== undefined ? "\n🩺 Health: " + data.health + "/100"
: "";

  // DNA info if present
  const dna = data.dna_summary ? "\n" + data.dna_summary : "";

  // Multi-TP levels
  const tp1Str = data.tp1 ? `\n🎯 <b>TP1:</b> <code>${data.tp1}</code>  RR
1:${data.rr1||"?"}  <i>→ 50%</i>` : "";
  const tp2Str = data.tp2 ? `\n🎯 <b>TP2:</b> <code>${data.tp2}</code>  RR
1:${data.rr2||"?"}  <i>→ 30%</i>` : "";
  const tp3Str = data.tp3 ? `\n🎯 <b>TP3:</b> <code>${data.tp3}</code>  RR
1:${data.rr3||"?"}  <i>→ 20%</i>` : "";

  const tpSection = (tp1Str || tp2Str || tp3Str)
    ? "━━━━━━━━━━━━━━━━━━━━━━\n🎯 <b>TARGETS</b>" + tp1Str + tp2Str + tp3Str + "\n"
    : `🎯 <b>TP:</b>     <code>${data.tp || "N/A"}</code>\n`;

  const text =
    `<b>🦉 ULUKA LIVE HOOT</b>\n` +
    `<b>Status:</b> ${icon} ${action} ORDER\n` +
    `━━━━━━━━━━━━━━━━━━━━━━\n` +
    `<b>Symbol:</b>   <code>${data.symbol  || "N/A"}</code>\n` +
    `<b>Strategy:</b> ${strat}\n` +
    (sess ? `<b>Session:</b>  ${sess}\n` : "") +
```

```
    (conf ? `<b>Confidence:</b> <code>${conf}%</code>\n` : "") +
    `<b>Time:</b>     <code>${time}</code>\n` +
    `━━━━━━━━━━━━━━━━━━━━━━━━\n` +
    `🔹 <b>ENTRY:</b>  <code>${data.entry || data.price || "N/A"}</code>\n` +
    `❌ <b>SL:</b>     <code>${data.sl    || "N/A"}</code>\n` +
    tpSection +
    `📦 <b>Lot:</b>    ${data.lot    || "N/A"}\n` +
    `🎫 <b>Ticket:</b> ${data.ticket || "-"}\n` +
    health + dna + "\n" +
    `\n<i>${DISCLAIMER}</i>`;

  const kb = {
    "inline_keyboard": [[
      { "text": "🛠️ Contact Support", "url": "https://t.me/UlukaOwlbot" },
      { "text": "👤 @WiseOwlUluka",  "url": "https://t.me/WiseOwlUluka" }
    ],[
      { "text": "🦉 Get Uluka Ultra", "url":
"https://docs.google.com/forms/d/e/1FAIpQLSfi3DU4Xe5BC3d4WV9y0fwjS5tgKOv3nVJKGJsv-yA
nDi9DnQ/viewform" }
    ]]
  };

  // NOTE: Premium group is sent directly by Master EA via SendToTelegram()
  // GAS only handles the Free group here to avoid double messages
   if (PREMIUM_GROUP_ID) sendToChat(PREMIUM_GROUP_ID, text, kb); // DISABLED -
Master sends this

  if (FREE_GROUP_ID) {
    const tp1Only = data.tp1
      ? `\n🎯 <b>TP1:</b> <code>${data.tp1}</code>  RR 1:${data.rr1||"?"}  <i>→
50%</i>`
      : (data.tp ? `\n🎯 <b>TP:</b> <code>${data.tp}</code>` : "");

    const freeText =
      `<b>🦉 ULUKA SIGNAL ALERT</b>\n` +
      `${icon} <b>${action}</b> on <code>${data.symbol || "N/A"}</code>\n` +
      `Strategy: ${strat}\n` +
      (sess ? `Session: ${sess}\n` : "") +
```

```
      `Time: ${time}\n` +
      `🔹 <b>Entry:</b> <code>${data.entry || data.price || "N/A"}</code>\n` +
      tp1Only + `\n\n` +
      `💎 <b>Full levels (SL + TP2 + TP3) in Premium</b>\n\n` +
      `<i>${DISCLAIMER}</i>`;

    sendToChat(FREE_GROUP_ID, freeText, {
      "inline_keyboard": [[
        { "text": "💎 Join Premium",    "url": "https://t.me/+I6NkMixMbnFmOTU1" },
        { "text": "🦉 Get Uluka Ultra", "url":
"https://docs.google.com/forms/d/e/1FAIpQLSfi3DU4Xe5BC3d4WV9y0fwjS5tgKOv3nVJKGJsv-yA
nDi9DnQ/viewform" }
      ]]
    });
  }
}

// ============================================================
// 9. GROUP ACTIVATION HOOT
// ============================================================

function sendGroupActivationHoot(data) {
  const msg =
    `🦉 <b>ULUKA IS LIVE</b>\n\n` +
    `A new trading session has been activated.\n` +
    `Expect live signals shortly.\n\n` +
    `<i>${DISCLAIMER}</i>`;

  const premKb = {
    "inline_keyboard": [[
      { "text": "🛠️ Contact Support", "url": "https://t.me/UlukaOwlbot" },
      { "text": "👤 @WiseOwlUluka",  "url": "https://t.me/WiseOwlUluka" }
    ],[
      { "text": "🦉 Get Uluka Ultra", "url":
"https://docs.google.com/forms/d/e/1FAIpQLSfi3DU4Xe5BC3d4WV9y0fwjS5tgKOv3nVJKGJsv-yA
nDi9DnQ/viewform" }
    ]]
  };
```

```
  const freeKb = {
    "inline_keyboard": [[
      { "text": "💎 Join Premium",    "url": "https://t.me/+I6NkMixMbnFmOTU1" },
      { "text": "🦉 Get Uluka Ultra", "url":
"https://docs.google.com/forms/d/e/1FAIpQLSfi3DU4Xe5BC3d4WV9y0fwjS5tgKOv3nVJKGJsv-yA
nDi9DnQ/viewform" }
    ]]
  };

  if (PREMIUM_GROUP_ID) sendToChat(PREMIUM_GROUP_ID, msg, premKb);
  if (FREE_GROUP_ID)    sendToChat(FREE_GROUP_ID,    msg, freeKb);
}

// ============================================================
// 10. GUARDIAN ANGEL ALERT
// ============================================================

function handleGuardianAlert(data) {
  const msg =
    `🚨 <b>GUARDIAN ANGEL EMERGENCY</b>\n\n` +
    `⚠️ Emergency drawdown triggered!\n\n` +
    `👤 <b>Client:</b> ${data.client  || "Unknown"}\n` +
    `🆔 <b>Account:</b> <code>${data.account || "?"}</code>\n` +
    `📉 <b>Drawdown:</b> ${data.dd     || "?"}%\n` +
    `💰 <b>Equity:</b> $${data.equity || "?"}\n` +
    `⏱️ <b>Time:</b> ${data.time      || new Date().toString()}\n\n` +
    `🛑 All positions closed. Trading locked.`;

  sendAdminAlert(msg);
  if (data.chat_id) sendToChat(data.chat_id, msg);
}

// ============================================================
// 11. PER-CLIENT EOD REPORT
// ============================================================

function sendClientEODReport(data) {
  const chatId = data.chat_id || data.personal_chat_id || "";
```

```
  if (!chatId) return;

  const pnlIcon    = parseFloat(data.total_pnl || 0) >= 0 ? "✅" : "🔴";
  const healthNum  = parseInt(data.health) || 0;
  const healthIcon = healthNum >= 70 ? "🟢" : healthNum >= 50 ? "🟡" : healthNum >=
30 ? "🟠" : "🔴";

  const msg =
    `🦉 <b>YOUR DAILY REPORT</b>\n` +
    `📅 ${data.date || Utilities.formatDate(new Date(), "GMT+5:30",
"dd-MMM-yyyy")}\n\n` +
    `👤 <b>${data.client || "Client"}</b>\n` +
    `🆔 Account: <code>${data.account || "N/A"}</code>\n\n` +
    `📊 <b>TODAY'S PERFORMANCE</b>\n` +
    `• Trades:   <code>${data.trades   || 0}</code>\n` +
    `• Wins:     <code>${data.wins     || 0}</code>  |  Losses: <code>${data.losses
|| 0}</code>\n` +
    `• Win Rate: <code>${data.win_rate || 0}%</code>\n\n` +
    `💰 <b>P&amp;amp;L</b>\n` +
    `• Realized: <code>$${parseFloat(data.realized  || 0).toFixed(2)}</code>\n` +
    `• Floating: <code>$${parseFloat(data.floating  || 0).toFixed(2)}</code>\n` +
    `• ${pnlIcon} Total: <b>$${parseFloat(data.total_pnl || 0).toFixed(2)}</b>\n\n`
+
    `🏦 <b>ACCOUNT</b>\n` +
    `• Balance: <code>$${data.balance || "N/A"}</code>\n` +
    `• Equity:  <code>$${data.equity  || "N/A"}</code>\n\n` +
    `${healthIcon} <b>Health Score:</b> ${data.health}/100\n` +
    (data.prop && data.prop !== "None" ? `🏆 <b>Prop Mode:</b> ${data.prop}\n` :
"") +
    `\n<i>${DISCLAIMER}</i>`;

  sendToChat(chatId, msg);
}

// ============================================================
// 12. ADMIN EOD REPORT
// ============================================================
```

```
function sendDailyEODReport() {
  const ss           = SpreadsheetApp.getActiveSpreadsheet();
  const billingSheet = ss.getSheetByName("Billing");
  const keySheet     = ss.getSheetByName(SHEET_NAME);

  if (!billingSheet || !keySheet) {
    sendAdminAlert("⚠️ EOD Report failed: Sheet not found.");
    return;
  }

  const bData   = billingSheet.getDataRange().getValues();
  const kData   = keySheet.getDataRange().getValues();
  const today   = new Date();
  const dateStr = Utilities.formatDate(today, "GMT+5:30", "dd-MMM-yyyy");

  let activeCount  = 0;
  let expiringSoon = [];

  for (let j = 1; j < kData.length; j++) {
    const status = (kData[j][COLUMNS.STATUS - 1] || "").toString();
    if (status.toUpperCase() !== "ACTIVE") continue;
    activeCount++;
    const expDate = new Date(kData[j][COLUMNS.EXPIRES_ON - 1]);
    const diff    = (expDate - today) / (1000 * 60 * 60 * 24);
    if (diff <= 3 && diff >= 0)
      expiringSoon.push(kData[j][0] + " (" + Math.ceil(diff) + "d left)");
  }

  let totalNetProfit = 0, topProfit = -Infinity, bestClient = "N/A";
  let maxDD = 0, riskyAcc = "None", detailsMsg = "";

  for (let i = 1; i < bData.length; i++) {
    const client = bData[i][0]  || "Unknown";
    const profit = parseFloat(bData[i][4]) || 0;
    const dd     = parseFloat(bData[i][8]) || 0;
    totalNetProfit += profit;
    if (profit > topProfit) { topProfit = profit; bestClient = client; }
```

```
    if (dd > maxDD)         { maxDD = dd; riskyAcc = client + " (" + dd.toFixed(1) +
"%)"; }
    detailsMsg += `\n${profit >= 0 ? "🟢":"🔴"} ${client.split(' ')[0]}:
<b>$${profit.toFixed(0)}</b>`;
  }

  let msg =
    `<b>📊 ULUKA MASTER EOD REPORT</b>\n📅 <i>${dateStr}</i>\n\n` +
    `💎 <b>PORTFOLIO</b>\n` +
    `• Active Clients: <code>${activeCount}</code>\n` +
    `• Total Net Profit: <b>$${totalNetProfit.toFixed(2)}</b>\n` +
    `• Your 25% Cut: <u><b>$${(totalNetProfit * 0.25).toFixed(2)}</b></u>\n\n` +
    `🌟 Top Gainer: ${bestClient}\n` +
    `⚠️ Highest DD: ${riskyAcc}\n`;

  if (expiringSoon.length > 0)
    msg += `\n⏳ <b>RENEWING SOON:</b>\n• ${expiringSoon.join("\n• ")}\n`;

  msg += `\n📝 <b>CLIENTS:</b><code>${detailsMsg}</code>\n\n✅ <i>EOD Complete –
${dateStr}</i>`;

  sendAdminAlert(msg);
}

// ============================================================
// 13. TELEGRAM CORE
// ============================================================

function sendToChat(chatId, text, keyboard) {
  if (!chatId || chatId === "" || chatId === "0") return false;
  if (!TELEGRAM_BOT_TOKEN) {
    console.error("TG_TOKEN not set.");
    return false;
  }

  try {
    const payload = {
      "chat_id":                  chatId.toString(),
```

```
      "text":                     text,
      "parse_mode":               "HTML",
      "disable_web_page_preview": true
    };
    if (keyboard) payload["reply_markup"] = JSON.stringify(keyboard);

    const res = UrlFetchApp.fetch(
      `https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/sendMessage`,
      {
        "method":             "post",
        "contentType":        "application/json",
        "payload":            JSON.stringify(payload),
        "muteHttpExceptions": true
      }
    );

    if (res.getResponseCode() !== 200) {
      console.error("sendToChat failed (" + chatId + "): " + res.getContentText());
      return false;
    }
    return true;

  } catch (err) {
    console.error("sendToChat exception: " + err.message);
    return false;
  }
}

function sendAdminAlert(msg) {
  return sendToChat(ADMIN_CHAT_ID, "🦉 Uluka Admin Alert triggered ");
}

function sendTestAlert() {
  const ok = sendAdminAlert(
    "✅ <b>System Online v5.1</b>\n" +
    "Uluka Activated\n" +
    "Time: " + Utilities.formatDate(new Date(), "GMT+5:30", "dd-MMM HH:mm")
  );
```

```
  SpreadsheetApp.getUi().alert(ok ? "✅ Test sent!" : "❌ Failed — check TG_TOKEN
in Script Properties.");
}

// ============================================================
// 14. EXPIRY CHECK
// ============================================================

function autoCheckExpiry() {
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName(SHEET_NAME);
  if (!sheet) return;

  const rows = sheet.getDataRange().getValues();

  for (let i = 1; i < rows.length; i++) {
    const expCell = rows[i][COLUMNS.EXPIRES_ON - 1];
    const status  = String(rows[i][COLUMNS.STATUS - 1]).toUpperCase();
    if (!expCell || status === "EXPIRED") continue;

    // v5.3 fix: handle Sheets Date objects and empty cells
    let expDate2;
    if (expCell instanceof Date && !isNaN(expCell.getTime())) {
      expDate2 = new Date(expCell);
    } else {
      const expRaw2 = (expCell || "").toString().trim();
      if (!expRaw2) continue; // skip if no expiry set
      const ep3 = expRaw2.split("-");
      expDate2 = (ep3.length === 3 && ep3[0].length === 2)
        ? new Date(ep3[2] + "/" + ep3[1] + "/" + ep3[0])
        : new Date(expRaw2);
    }
    if (isNaN(expDate2.getTime())) continue; // skip invalid dates
    if (expDate2 < new Date()) {
      sheet.getRange(i + 1, COLUMNS.STATUS).setValue("EXPIRED");
      sendAdminAlert(
        `⏰ <b>LICENCE EXPIRED</b>\n` +
        `👤 ${rows[i][0]}\n` +
        `🔑 <code>${rows[i][COLUMNS.LICENCE_KEY - 1]}</code>`
```

```
      );
    }
  }
}

// ==============================================================
// 15. FRIDAY PAYE ARCHIVE
// ==============================================================

function runFridayPayeArchive() {
  const ss     = SpreadsheetApp.getActiveSpreadsheet();
  const bSheet = ss.getSheetByName("Billing");
  if (!bSheet) return;

  let payeSheet = ss.getSheetByName("PAYE");
  if (!payeSheet) {
    payeSheet = ss.insertSheet("PAYE");
    payeSheet.appendRow(["Date","Client","Account ID","Payee Amount","Status","Net
Profit","Period"]);
  }

  const bData  = bSheet.getDataRange().getValues();
  const today  = Utilities.formatDate(new Date(), "GMT+5:30", "dd-MMM-yyyy");
  let count = 0, total = 0;

  for (let i = 1; i < bData.length; i++) {
    const netProfit = parseFloat(bData[i][3]) - parseFloat(bData[i][2]);
    const payeeAmt  = netProfit > 0 ? netProfit * 0.25 : 0;
    if (payeeAmt <= 0) continue;

    payeSheet.appendRow([today, bData[i][0], bData[i][1],
                         payeeAmt.toFixed(2), "🟡 Pending",
                         netProfit.toFixed(2), "Weekly"]);

    bSheet.getRange(i+1, 3).setValue(parseFloat(bData[i][3]));
    bSheet.getRange(i+1, 5).setFormula(`=D${i+1}-C${i+1}`);
    bSheet.getRange(i+1, 6).setFormula(`=IF(E${i+1}>0,E${i+1}*0.25,0)`);
    count++; total += payeeAmt;
```

```
  }

  SpreadsheetApp.flush();

  sendAdminAlert(count > 0
    ? `💰 <b>FRIDAY PAYE</b>\n✅ ${count} accounts\n💵 Total:
<b>$${total.toFixed(2)}</b>\n📅 ${today}`
    : `📊 <b>Friday Archive:</b> No profit to archive.`
  );
}

// ============================================================
// 16. KEY GENERATOR
// ============================================================

function showKeyGeneratorPrompt() {
  const ui = SpreadsheetApp.getUi();

  const nameR = ui.prompt("STEP 1 — Client Name", "Full name:",
ui.ButtonSet.OK_CANCEL);
  if (nameR.getSelectedButton() !== ui.Button.OK) return;
  const name = nameR.getResponseText().trim();
  if (!name) { ui.alert("Name cannot be empty."); return; }

  const subR = ui.prompt("STEP 2 — Subscription", "PRO / TRIAL / MONTHLY / PAYE:",
ui.ButtonSet.OK_CANCEL);
  if (subR.getSelectedButton() !== ui.Button.OK) return;
  const subType = subR.getResponseText().toUpperCase().trim() || "MONTHLY";

  const durR = ui.prompt("STEP 3 — Duration", "Days (e.g. 30):",
ui.ButtonSet.OK_CANCEL);
  if (durR.getSelectedButton() !== ui.Button.OK) return;
  const days = parseInt(durR.getResponseText()) || 30;

  const capR = ui.prompt("STEP 4 — Equity Cap", "Max equity $ (e.g. 2000):",
ui.ButtonSet.OK_CANCEL);
  if (capR.getSelectedButton() !== ui.Button.OK) return;
  const cap = capR.getResponseText().trim() || "500";
```

```
  const key       = "ULUKA-" + Math.random().toString(36).substring(2,
10).toUpperCase();
  const createdOn = new Date();
  const expiresOn = new Date(createdOn);
  expiresOn.setDate(createdOn.getDate() + days);

  const cleanCreated = Utilities.formatDate(createdOn, "GMT", "dd-MM-yyyy");
  const cleanExpiry  = Utilities.formatDate(expiresOn, "GMT", "dd-MM-yyyy");

  const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName(SHEET_NAME);
  sheet.appendRow([name, key, "", "", cleanCreated, subType,
                   days + " Days", cleanExpiry, "ACTIVE", cap, "", "NO", 0, "",
""]);

  sendAdminAlert(
    `🔑 <b>NEW KEY</b>\n👤 ${name}\n🔑 <code>${key}</code>\n` +
    `📦 ${subType} — ${days} days\n💰 Cap: $${cap}\n📅 Expires: ${cleanExpiry}`
  );

  ui.alert("✅ Key Generated!\n\nKey: " + key + "\nExpires: " + cleanExpiry);
}

// ============================================================
// 17. UTILITIES
// ============================================================

function clearHwidData() {
  const row = SpreadsheetApp.getActiveSpreadsheet().getActiveCell().getRow();
  if (row <= 1) { SpreadsheetApp.getUi().alert("Select a data row first."); return;
}
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName(SHEET_NAME);
  sheet.getRange(row, COLUMNS.HWID).clearContent();
  sheet.getRange(row, COLUMNS.ACTIVATIONS).setValue(0);
  sheet.getRange(row, COLUMNS.INSTANCE_ID).clearContent();
  SpreadsheetApp.getUi().alert("✅ HWID cleared for row " + row);
}
```

```
function logRateLimit(ip, account) {
  try {
    const ss    = SpreadsheetApp.getActiveSpreadsheet();
    const sheet = ss.getSheetByName("RateLimit") || ss.insertSheet("RateLimit");
    sheet.appendRow([new Date(), ip, account, "OK"]);
  } catch(e) {}
}

// ============================================================
// 18. DASHBOARD LIVE DATA API
// ============================================================

function getDashboardData() {
  try {
    const ss        = SpreadsheetApp.getActiveSpreadsheet();
    const keySheet  = ss.getSheetByName(SHEET_NAME);
    const billSheet = ss.getSheetByName("Billing");
    const tradeSheet= ss.getSheetByName("TradeLog");
    const today     = new Date();

    // —— Per-client data ——
    const rows = keySheet.getDataRange().getValues();
    let activeClients = 0, totalNetProfit = 0, totalWins = 0, totalTrades = 0;
    let openPositions = 0;
    const clients = [];

    for (let i = 1; i < rows.length; i++) {
      if (!rows[i][0]) continue;
      const status = String(rows[i][COLUMNS.STATUS - 1] || "").toUpperCase();
      if (status !== "ACTIVE" && status !== "AUTHORIZED") continue;
      activeClients++;

      const name    = rows[i][0] || "Client";
      const account = String(rows[i][COLUMNS.ACCOUNT_ID - 1] || "");

      // Expiry
      const rawExpVal = rows[i][COLUMNS.EXPIRES_ON - 1];
      let expDate;
```

```
      if (rawExpVal instanceof Date && !isNaN(rawExpVal.getTime())) {
        expDate = rawExpVal;
      } else {
        const rawExpStr = (rawExpVal || "").toString().trim();
        const ep2 = rawExpStr.split("-");
        expDate = (ep2.length === 3 && ep2[0].length === 2)
          ? new Date(ep2[2] + "/" + ep2[1] + "/" + ep2[0])
          : new Date(rawExpStr || today);
      }
      if (isNaN(expDate.getTime())) expDate = new Date(today.getTime() +
365*86400000);
      const daysLeft = Math.max(0, Math.ceil((expDate - today) / 86400000));

      // Billing data for this client
      let balance = 0, profit = 0, dd = 0, wins = 0, losses = 0;
      if (billSheet) {
        const bData = billSheet.getDataRange().getValues();
        for (let j = 1; j < bData.length; j++) {
          if (String(bData[j][1]) === account || bData[j][0] === name) {
            balance = parseFloat(bData[j][2]) || 0;
            profit  = parseFloat(bData[j][4]) || 0;
            dd      = parseFloat(bData[j][8]) || 0;
            wins    = parseInt(bData[j][5])   || 0;
            losses  = parseInt(bData[j][6])   || 0;
            break;
          }
        }
      }

      const totalC = wins + losses;
      const wr     = totalC > 0 ? Math.round((wins / totalC) * 100) : 0;
      const hasData = (balance > 0 ||  totalC > 0);
      const health = !hasData ? 0 : Math.min(100, Math.max(0, Math.round(
        (wr * 0.4) + (dd < 2 ? 40 : dd < 4 ? 25 : dd < 6 ? 10 : 0) + (profit > 0 ?
20 : 0)
      )));

      totalNetProfit += profit;
```

```
      totalWins      += wins;
      totalTrades    += totalC;

      clients.push({
        name:    name.split(" ")[0] + (name.split(" ")[1] ? " " + name.split(" 
")[1][0] + "." : ""),
        account: account || "N/A",
        balance: balance,
        profit:  profit,
        dd:      dd.toFixed(1) + "%",
        status:  daysLeft < 3 ? "warning" : "active",
        health:  health,
        expiry:  daysLeft + "d"
      });
    }

    // ── Recent trades from TradeLog ──
    const recentTrades = [];
    if (tradeSheet) {
      const tData = tradeSheet.getDataRange().getValues();
      const start = Math.max(1, tData.length - 10);
      for (let i = start; i < tData.length; i++) {
        recentTrades.push({
          time:     tData[i][0] ? Utilities.formatDate(new Date(tData[i][0]), "GMT", 
"HH:mm") : "--:--",
          account:  tData[i][1],
          symbol:   tData[i][2]  || "N/A",
          action:   tData[i][3]  || "BUY",
          profit:   parseFloat(tData[i][4]) || 0,
          lot:      parseFloat(tData[i][5]) || 0.01,
          strategy: tData[i][6]  || "AI"
        });
      }
    }

    // ── Billing summary ──
    let weeklyProfit = 0, totalEarned = 0, pendingPAYE = 0;
    if (billSheet) {
```

```
      const bData = billSheet.getDataRange().getValues();
      for (let i = 1; i < bData.length; i++) {
        const profit = parseFloat(bData[i][4]) || 0;
        weeklyProfit += profit;
        totalEarned  += parseFloat(bData[i][5]) || 0;
      }
      pendingPAYE = weeklyProfit * 0.25;
    }

    // —— Next Friday ——
    const nextFriday = new Date(today);
    const dayOfWeek  = nextFriday.getDay();
    const daysToFri  = (5 - dayOfWeek + 7) % 7 || 7;
    nextFriday.setDate(nextFriday.getDate() + daysToFri);
    const nextPAYEStr = Utilities.formatDate(nextFriday, "GMT", "dd MMM");

    const avgWinRate = totalTrades > 0 ? Math.round((totalWins / totalTrades) * 100)
: 0;
    const yourCut    = totalNetProfit * 0.25;

    const payload = {
      ok: true,
      generated: new Date().toISOString(),
      stats: {
        activeClients:   activeClients,
        totalNetProfit:  parseFloat(totalNetProfit.toFixed(2)),
        yourCut:         parseFloat(yourCut.toFixed(2)),
        avgWinRate:      avgWinRate,
        tradesThisWeek:  totalTrades,
        openPositions:   openPositions
      },
      clients: clients,
      recentTrades: recentTrades.reverse(),
      billing: {
        weeklyProfit: parseFloat(weeklyProfit.toFixed(2)),
        weeklyPayee:  parseFloat((weeklyProfit * 0.25).toFixed(2)),
        pendingPAYE:  parseFloat(pendingPAYE.toFixed(2)),
        totalEarned:  parseFloat(totalEarned.toFixed(2)),
```

```
        nextPAYE:     nextPAYEStr
      }
    };

    return payload;  // ← FIX 1: was missing, return was inside catch

  } catch(e) {
    console.error("getDashboardData error: " + e.message);
    return { ok: false, error: e.message };  // ← FIX 2: was ContentService
  }
}

function getDashboardDataJSON() {
  try {
    const ss         = SpreadsheetApp.getActiveSpreadsheet();
    const keySheet   = ss.getSheetByName(SHEET_NAME);
    const billSheet  = ss.getSheetByName("Billing");
    const tradeSheet = ss.getSheetByName("TradeLog");
    const today      = new Date();

    const rows = keySheet.getDataRange().getValues();
    let activeClients = 0, totalNetProfit = 0, totalWins = 0, totalTrades = 0;
    const clients = [];

    for (let i = 1; i < rows.length; i++) {
      if (!rows[i][0]) continue;
      const status = String(rows[i][COLUMNS.STATUS - 1] || "").toUpperCase();
      if (status !== "ACTIVE" && status !== "AUTHORIZED") continue;
      activeClients++;

      const name    = rows[i][0] || "Client";
      const account = String(rows[i][COLUMNS.ACCOUNT_ID - 1] || "");

      const rawExpVal = rows[i][COLUMNS.EXPIRES_ON - 1];
      let expDate;
      if (rawExpVal instanceof Date && !isNaN(rawExpVal.getTime())) {
        expDate = rawExpVal;
      } else {
```

```
        const ep2 = (rawExpVal || "").toString().trim().split("-");
        expDate = (ep2.length === 3 && ep2[0].length === 2)
          ? new Date(ep2[2] + "/" + ep2[1] + "/" + ep2[0])
          : new Date(rawExpVal || today);
      }
      if (isNaN(expDate.getTime())) expDate = new Date(today.getTime() +
365*86400000);
      const daysLeft = Math.max(0, Math.ceil((expDate - today) / 86400000));

      let balance = 0, profit = 0, dd = 0, wins = 0, losses = 0;
      if (billSheet) {
        const bData = billSheet.getDataRange().getValues();
        for (let j = 1; j < bData.length; j++) {
          if (String(bData[j][1]) === account || bData[j][0] === name) {
            balance = parseFloat(bData[j][3]) || 0;
            profit  = parseFloat(bData[j][4]) || 0;
            dd      = parseFloat(bData[j][8]) || 0;
            wins    = 0;
            losses  = 0;
            break;
          }
        }
      }

      const totalC = wins + losses;
      const wr     = totalC > 0 ? Math.round((wins / totalC) * 100) : 0;
      const hasData = (balance > 0  ||  totalC > 0);
      const health = !hasData ? 0 : Math.min(100, Math.max(0, Math.round(
        (wr * 0.4) + (dd < 2 ? 40 : dd < 4 ? 25 : dd < 6 ? 10 : 0) + (profit > 0 ?
20 : 0)
      )));

      totalNetProfit += profit;
      totalWins      += wins;
      totalTrades    += totalC;

      clients.push({
```

```
        name:    name.split(" ")[0] + (name.split(" ")[1] ? " " + name.split(" ")[1][0] + "." : ""),
        account: account || "N/A",
        balance: balance,
        profit:  profit,
        dd:      dd.toFixed(1) + "%",
        status:  daysLeft < 3 ? "warning" : "active",
        health:  health,
        expiry:  daysLeft + "d"
      });
    }

    const recentTrades = [];
    if (tradeSheet) {
      const tData = tradeSheet.getDataRange().getValues();
      const start = Math.max(1, tData.length - 10);
      for (let i = start; i < tData.length; i++) {
        recentTrades.push({
          time:     tData[i][0] ? Utilities.formatDate(new Date(tData[i][0]), "GMT", "HH:mm") : "--:--",
          account:  String(tData[i][1] || ""),
          symbol:   tData[i][2]  || "N/A",
          action:   tData[i][3]  || "BUY",
          profit:   parseFloat(tData[i][4]) || 0,
          lot:      parseFloat(tData[i][5]) || 0.01,
          strategy: tData[i][6]  || "AI"
        });
      }
    }

    let weeklyProfit = 0, totalEarned = 0, pendingPAYE = 0;
    if (billSheet) {
      const bData = billSheet.getDataRange().getValues();
      for (let i = 1; i < bData.length; i++) {
        weeklyProfit += parseFloat(bData[i][4]) || 0;
        totalEarned  += parseFloat(bData[i][3]) || 0;
      }
      pendingPAYE = weeklyProfit * 0.25;
```

```
    }

    const nextFriday = new Date(today);
    const daysToFri  = (5 - nextFriday.getDay() + 7) % 7 || 7;
    nextFriday.setDate(nextFriday.getDate() + daysToFri);
    const nextPAYEStr = Utilities.formatDate(nextFriday, "GMT", "dd MMM");

    const avgWinRate = totalTrades > 0 ? Math.round((totalWins / totalTrades) * 100)
: 0;
    const yourCut    = totalNetProfit * 0.25;

    const payload = {
      ok: true,
      generated: new Date().toISOString(),
      stats: {
        activeClients:  activeClients,
        totalNetProfit: parseFloat(totalNetProfit.toFixed(2)),
        yourCut:        parseFloat(yourCut.toFixed(2)),
        avgWinRate:     avgWinRate,
        tradesThisWeek: totalTrades,
        openPositions:  0
      },
      clients:      clients,
      recentTrades: recentTrades.reverse(),
      billing: {
        weeklyProfit: parseFloat(weeklyProfit.toFixed(2)),
        weeklyPayee:  parseFloat((weeklyProfit * 0.25).toFixed(2)),
        pendingPAYE:  parseFloat(pendingPAYE.toFixed(2)),
        totalEarned:  parseFloat(totalEarned.toFixed(2)),
        nextPAYE:     nextPAYEStr
      }
    };

    return payload;  // ← FIX 3: was ContentService, now plain object

  } catch(e) {
    console.error("getDashboardDataJSON error: " + e.message);
    return { ok: false, error: e.message };  // ← FIX 4: was ContentService
```

```
  }
}

// Trade log to sheet //

function logTradeToSheet(p) {
  try {
    const ss = SpreadsheetApp.getActiveSpreadsheet();
    let sheet = ss.getSheetByName("TradeLog");
    if (!sheet) {
      sheet = ss.insertSheet("TradeLog");

sheet.appendRow(["Time","Account","Symbol","Action","Profit","Lot","Strategy"]);
    }
    sheet.appendRow([
      new Date(),
      String(p.account_id || p.account || ""),
      p.symbol   || "",
      p.action   || "",
      parseFloat(p.price  || 0),   // price = P&L in TRADE_LOG calls
      parseFloat(p.lot    || 0),
      p.strategy || p.type || ""
    ]);
  } catch(e) {
    console.error("logTradeToSheet error: " + e.message);
  }
}
```